

Open Redirect

Open Redirect is a deceptively simple vulnerability class — on its own it often gets dismissed as "low severity," but it's genuinely one of the best examples in this entire series of a bug whose real danger comes almost entirely from **chaining** it with something else (phishing, SSRF filter bypass, OAuth token theft). It also connects directly back to your XXE guide's SSRF discussion — one of the labs you'd encounter uses an open redirect specifically to defeat an SSRF host-restriction filter.

1. What Open Redirect Actually Is

Open redirection happens when an application takes user-controllable input and uses it — unsafely — as the destination of a redirect, without validating that the destination is actually an intended, trusted location. An attacker crafts a link that *appears* to point at the legitimate, trusted site, but actually redirects the victim onward to an attacker-controlled destination.

Legitimate-looking URL:

`https://trusted-website.com/redirect?url=https://attacker-website.com`

Victim clicks it (trusts the domain shown) → browser is sent to trusted-website.com
→ trusted-website.com's redirect logic reads the `url` parameter → sends the victim onward to attacker-website.com

Why this is dangerous despite "just" being a redirect: the victim's browser genuinely did visit the trusted domain first — this lends real credibility to the link (correct domain, valid TLS certificate, no obvious red flags in the URL bar at a glance) that a raw link straight to `attacker-website.com` would never have.

2. Server-Side vs. DOM-Based Open Redirection

This is the same server-side/client-side distinction that shows up across your series (compare: reflected vs. DOM-based XSS) — worth keeping the two mentally separate since detection and fixing differ.

a) Server-Side Open Redirect

The server itself reads a parameter and issues an HTTP redirect response (a `3xx` status with a `Location` header) pointing at attacker-controlled data:

```
GET /redirect?url=https://attacker.com HTTP/1.1

HTTP/1.1 302 Found
Location: https://attacker.com
```

b) DOM-Based Open Redirect

No server-side redirect happens at all — a **client-side script** reads attacker-controllable data (often straight from the URL, sometimes from `location.hash` which never even gets sent to the server) and writes it into a sink that triggers cross-domain navigation:

```
let url = /https?:\/\/\.[+/.exec(location.hash);
if (url) {
    location = url[0];
}
```

This code is vulnerable because it takes whatever URL-shaped string appears after the `#` in the current page's URL and blindly assigns it to `location`, sending the browser there — with zero server involvement, zero validation, and (since it's reading `location.hash`) the malicious destination never even shows up in server logs, since browsers don't send the fragment portion of a URL to the server at all.

3. Practical Example — Full Walkthrough

Legitimate application behavior (a "back to blog" link on an article page):
`https://vulnerable-website.com/post?postId=4&url=https://vulnerable-website.com/blog`

The page's client-side script reads the ``url`` parameter and, if it starts with `http://` or `https://`, uses it directly as the link's destination — with no further validation performed on where that destination actually points.

Attack:
`https://vulnerable-website.com/post?postId=4&url=https://attacker-website.com/`

The "Back to Blog" link on this page now silently points at the attacker's

site instead, while the URL bar still shows the trusted vulnerable-website.com domain right up until the click actually happens.

This exact pattern — a parameter whose only validation is checking the scheme prefix (`http://` / `https://`) and nothing else about the actual destination — is the single most common root cause you'll encounter in real applications and CTF labs alike.

4. Finding Open Redirects — What to Look For

Redirect functionality shows up constantly in legitimate app features, which is exactly why this vulnerability is so common:

Login/logout flows	→ ?returnUrl=/dashboard (redirect back after auth)
"Back to X" links	→ ?url=..., ?next=..., ?redirect=...
Language/locale switchers	→ ?url=... to return to the same page after switching
Tracking/analytics redirectors	→ /click?url=https://destination.com
Logout confirmation pages	→ ?continue=...
SSO/OAuth callback flows	→ redirect_uri parameters (covered in Section 6)

Common parameter names worth specifically testing on any app:

url, redirect, redirect_uri, redirect_url, return, returnUrl, return_to, next, dest, destination, target, continue, out, view, redir, forward

Detection methodology:

1. Identify any parameter that appears to control a redirect destination
2. Substitute a value pointing at a domain you control:
`?url=https://your-test-domain.com`
3. Follow the link (or check the Location header directly in Burp/ZAP without following redirects) and confirm you actually land on your domain, not just that the app accepted the parameter silently
4. If a straightforward full URL doesn't work, test the bypass techniques in Section 5 against whatever filtering is in place

5. Common Filter Bypass Techniques

Applications often attempt *some* validation — usually checking that the destination "starts with" the trusted domain, or requires a leading slash for local paths. These checks are frequently bypassable:

Expected safe pattern: destination must start with "/" (a relative, local path)

Bypass: `//attacker.com`

- browsers interpret a leading `//` as protocol-relative, and will happily navigate to `attacker.com` even though it "starts with `/`"

Expected safe pattern: destination must contain "trusted-website.com"

Bypass: `https://attacker.com/trusted-website.com`

- the string exists in the URL, but doesn't mean what a naive substring check assumed

Bypass: `https://trusted-website.com.attacker.com`

- the trusted domain appears as a SUBDOMAIN prefix of the attacker's actual domain, passing a naive "starts with" check

Expected safe pattern: destination must NOT contain "http"

Bypass: `Http://attacker.com` or URL-encoded variants

- case sensitivity and encoding are frequently overlooked in blocklist-style filters – the same blocklist weakness called out repeatedly across your SQLi/XXE/command injection guides

Expected safe pattern: only a bare hostname is compared

Bypass: `attacker.com@trusted-website.com`

- some URL parsers treat everything before the "@" as userinfo, and the ACTUAL host is what follows it

Every one of these bypasses exists because the developer validated against a **pattern** (starts-with, contains, doesn't-contain) instead of properly parsing the URL and checking the actual resolved hostname against a strict allowlist — precisely the same blocklist-vs-allowlist lesson from every other injection guide in this series, just applied to URL parsing instead of SQL/shell/XML syntax.

6. Why Open Redirect Is Rarely "Just" Low Severity — Chaining Examples

This is the section that matters most for understanding why bug bounty programs and VAPT reports take this seriously despite its simple mechanics.

a) Phishing Credibility Boost

The most straightforward chain: send a phishing link using the vulnerable trusted domain as the visible/clicked link, redirecting to a convincing fake login page. Security-aware users who've been trained to "check the domain before clicking" are specifically defeated by this, since the domain they see and click genuinely is the real, trusted one.

b) SSRF Filter Bypass (directly connects to your XXE guide)

Some applications restrict a server-side request feature (like a "check stock" function that fetches data from an internal system) to only accept URLs pointing at trusted/internal hosts. If that same application has an *unrelated* open redirect elsewhere, an attacker can point the restricted SSRF-prone feature at the **open redirect URL** instead of directly at the forbidden target:

1. stockApi parameter only accepts URLs on the app's own trusted host
2. The app also has an open redirect: /product/nextProduct?path=<destination>
3. Attacker sets: stockApi=/product/nextProduct?path=http://192.168.0.12:8080/admin
4. The app's own trusted-host check passes (the URL IS on the trusted host)
5. The app then follows ITS OWN redirect to the internal admin address, completely defeating the SSRF host restriction

This is a genuinely elegant technique worth remembering — a "low severity" open redirect became the missing piece that turned a properly-defended SSRF feature into a full internal-network-reaching primitive.

c) OAuth Token Theft via **redirect_uri** Abuse

OAuth flows rely on a **redirect_uri** parameter to tell the authorization server where to send the resulting access token/authorization code after the user grants consent. If the authorization server's validation of **redirect_uri** is weak (matches based on prefix/substring rather than exact allowlisted values) and the legitimate client application *also* happens to have an open redirect on one of its own pages, an attacker can:

1. Set redirect_uri to a legitimate-looking URL on the real client app that happens to also be an open redirect endpoint
2. The authorization server's loose validation accepts it (it IS the legitimate client's domain)
3. After the victim authenticates and consents, the OAuth token gets sent to the legitimate app's redirect endpoint...
4. ...which immediately open-redirects it onward to the attacker's server, handing over the victim's OAuth access token

This chain is a good illustration of why "the domain matches" is never sufficient validation on its own — the *specific path/endpoint* matters just as much as the domain itself.

7. Practical Testing With Burp/ZAP

1. Proxy your browsing through ZAP/Burp (per your OWASP ZAP guide)
2. Identify any redirect-related parameter from Section 4's naming list
3. Send the request to Repeater/Manual Request Editor
4. Substitute the parameter with a domain you control, or one of the Section 5 bypass patterns if basic substitution is filtered
5. Check the raw response's Location header directly - don't rely on your browser auto-following the redirect, since you want to confirm the EXACT destination the server intends to send you to
6. If chaining toward SSRF (Section 6b), test whether any server-side-request feature in the app can be pointed at your confirmed open-redirect endpoint instead of directly at a restricted target

8. Prevention — Fixing Open Redirect Properly

Best fix: avoid putting user-controllable data into a redirect destination at all.

Where redirect functionality genuinely requires dynamic destinations (post-login return URLs, "back to X" links):

a) Allowlist validation against a fixed set of permitted destinations — rather than trying to pattern-match/parse an arbitrary attacker-supplied URL safely (a task that's consistently proven error-prone, per every bypass in Section 5), maintain an explicit list of exactly which destinations are valid and reject anything not on it.

b) Use indirect references instead of raw URLs — instead of a `url=` parameter accepting a literal destination, use a short, server-side-mapped identifier (`redirect=2` meaning "the dashboard," `redirect=3` meaning "the profile page") that the server resolves internally — the user never controls the actual destination string at all, eliminating the entire attack surface the same way indirect object references mitigate IDOR from your earlier guide.

c) For DOM-based cases, apply the same allowlist logic client-side — since the vulnerable code runs entirely in the browser, the validation has to happen in that same client-side script, checking the destination against a permitted list before ever assigning it to `location` / `window.location`.

d) Require explicit user confirmation before redirecting off-site — an interstitial "you are leaving this site, continue?" page that clearly displays the actual destination doesn't prevent the technical vulnerability, but substantially reduces the phishing-credibility exploit from Section 6a, since it breaks the illusion of a seamless, fully-trusted redirect.

9. Practical Lab Example

PortSwigger's Web Security Academy has dedicated labs for both server-side and DOM-based open redirection, plus the SSRF-filter-bypass-via-open-redirect lab referenced in Section 6b – genuinely the most directly applicable hands-on environment for this specific topic, since it walks through the exact chaining technique in a realistic setting.

Basic practice flow:

1. Find the redirect parameter on a lab/test app
2. Confirm basic redirection to an external test domain works
3. If filtering exists, work through Section 5's bypass techniques systematically until one succeeds
4. If the app also has any server-side-request feature, specifically test whether it can be pointed at your confirmed redirect endpoint to reach an otherwise-restricted internal destination

10. Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-601 (URL Redirection to Untrusted Site, "Open Redirect")
OWASP Top 10:2025	A01:2025 Broken Access Control (when chained into SSRF, per your OWASP Top 10:2025 guide's note that SSRF was absorbed here) — standalone open redirect is also frequently discussed under general Security Misconfiguration (A02:2025) depending on context
CVSS	Standalone open redirect typically scores Low/Medium — low direct impact on its own; severity climbs sharply once chained (SSRF, OAuth token theft) since the CVSS score should reflect the actual achievable impact of the full chain, not just the redirect in isolation
Cyber Kill Chain	Delivery (the phishing-credibility use case) or Reconnaissance/Exploitation (the SSRF-chaining use case)
MITRE ATT&CK	T1566 (Phishing) when used for the phishing-credibility chain; supports T1190 (Exploit Public-Facing Application) when chained into SSRF

Quick Reference Cheat Sheet

Detection:

?url=https://your-test-domain.com

← basic substitution test

Check the raw Location header, don't rely on auto-follow

Common parameter names to test:

url, redirect, redirect_uri, return, returnUrl, next, dest,
target, continue, out, forward

Bypass techniques when filtered:

//attacker.com	← protocol-relative
https://trusted.com.attacker.com	← subdomain trick
attacker.com@trusted.com	← userinfo trick
HtTp://attacker.com	← case variation

Chaining:

Phishing → credible-looking trusted-domain link

SSRF bypass → point a restricted feature at your open-redirect URL

OAuth token theft → redirect_uri pointing at a client app's own open redirect

Fix:

Allowlist exact destinations, or use indirect server-side-mapped IDs
instead of raw attacker-controlled URLs